

TASK 2 — WEB APPLICATION VULNERABILITY ASSESSMENT

1. Objective

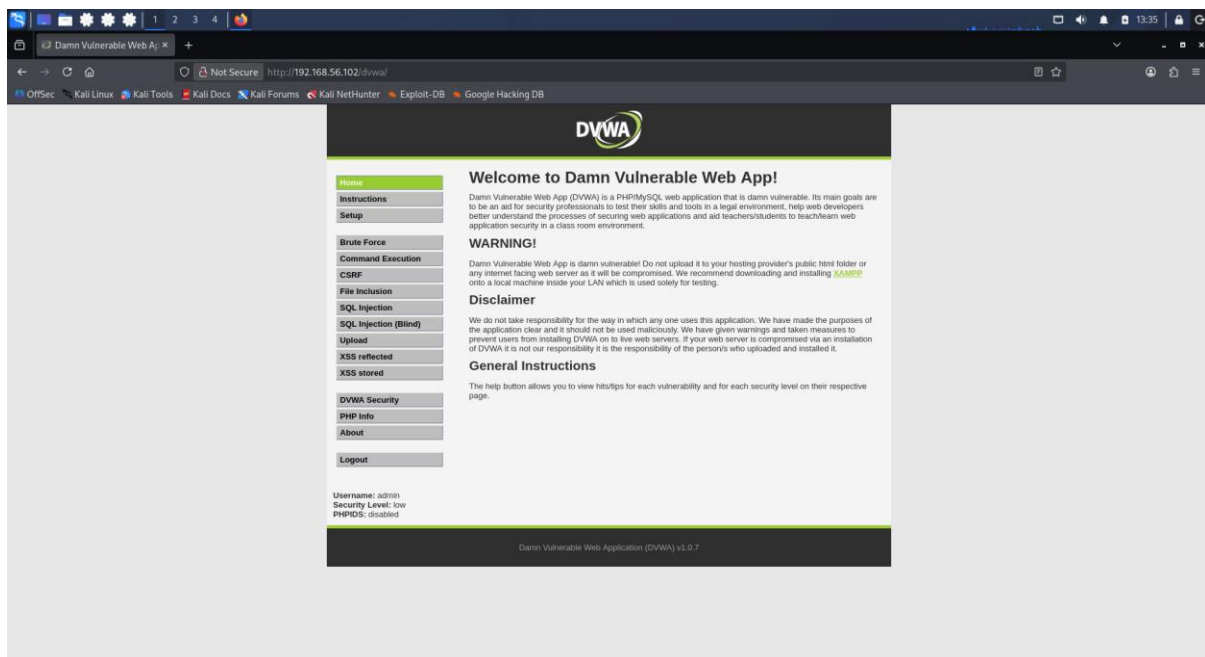
The objective of Task 2 was to perform a practical security assessment of a deliberately vulnerable web application using **DVWA (Damn Vulnerable Web Application)** and **Burp Suite**.

The assessment focused on identifying and demonstrating web application vulnerabilities by:

- Intercepting HTTP requests using Burp Suite.
- Examining HTTP requests and responses.
- Testing SQL Injection.
- Testing the effect of different DVWA security levels.
- Testing Reflected Cross-Site Scripting (XSS).
- Observing application and database error behavior.
- Collecting screenshots and HTTP evidence for documentation.

The testing was performed in an isolated virtual lab environment using the DVWA target at:

<http://192.168.56.102/dvwa/>



2. Lab Environment

Component	Details
Attacker/Test Machine	Kali Linux
Web Security Tool	Burp Suite Community Edition
Browser	Mozilla Firefox
Vulnerable Application	DVWA
DVWA Version	1.0.7
Target IP	192.168.56.102
Web Server	Apache
Backend	PHP/MySQL
Network	Isolated virtual lab
Security Level Tested	Low, Medium, High

The screenshots collected during the assessment show communication between the Kali Linux system and the DVWA machine over the local virtual network.

3. DVWA Configuration

After accessing the DVWA application, the application was successfully loaded and authenticated.

The DVWA Security page was examined to understand the available security levels.

DVWA provides three relevant security levels:

Low

Medium

High

During testing, the security level was changed to observe how application-side protections affected the same vulnerability.

Initial configuration

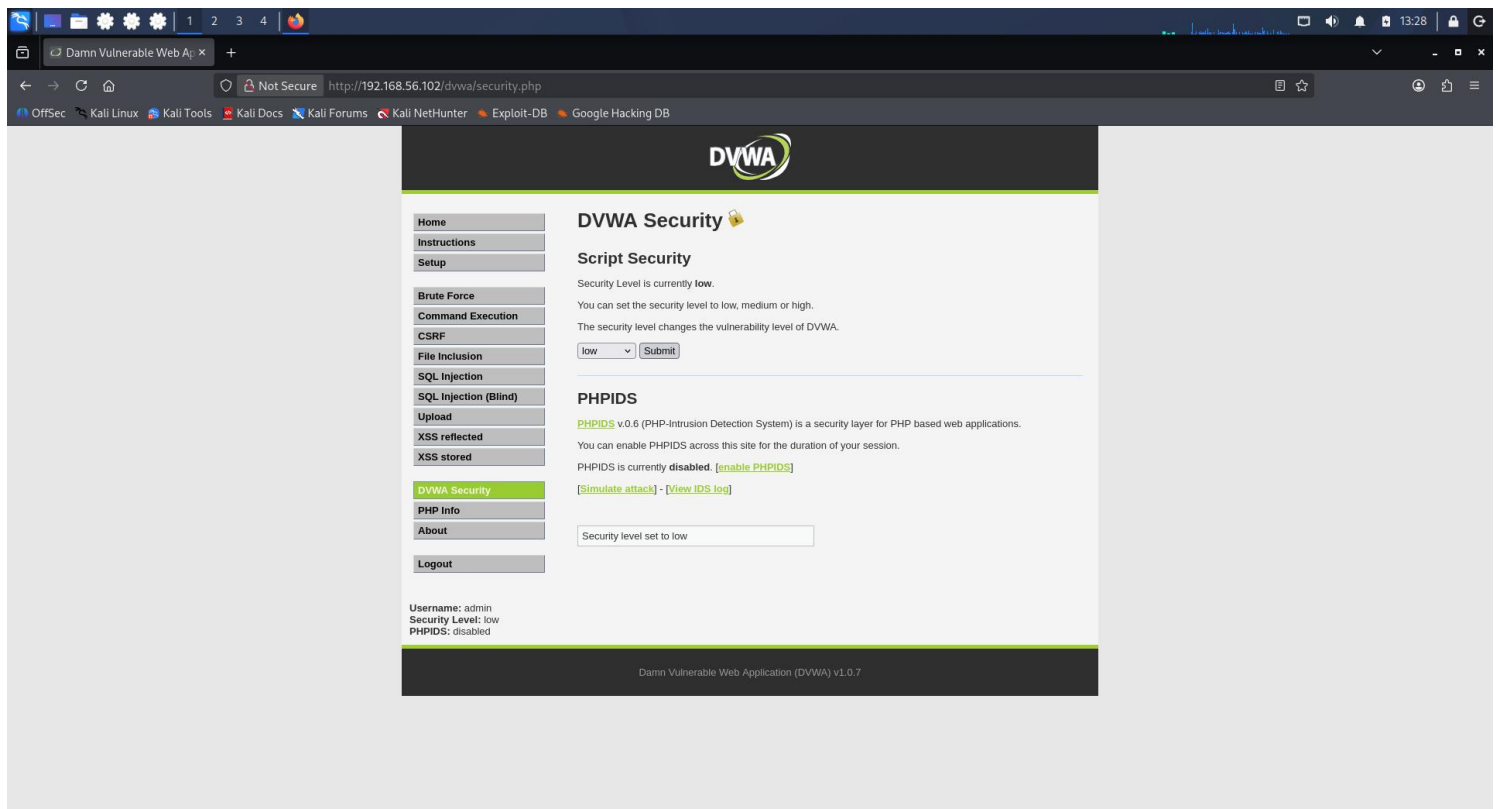
The application initially operated at:

Security Level: Low

PHPIDS: Disabled

The security configuration was subsequently changed to **Medium** and **High** during testing.

Evidence



4. Burp Suite Configuration

Burp Suite was used as the HTTP interception and analysis tool.

The browser traffic was routed through Burp Suite's proxy listener.

The Burp Suite HTTP history showed requests being generated by Firefox when accessing DVWA.

Example target:

192.168.56.102

Example request:

GET /dvwa/ HTTP/1.1

Host: 192.168.56.102

Burp Suite successfully captured the requests and responses.

This confirmed that the testing environment was correctly configured for web application traffic analysis.

5. HTTP Request and Response Analysis

Burp Suite was used to examine the structure of the HTTP communication.

A typical request contained information such as:

GET /dvwa/vulnerabilities/sqli/ HTTP/1.1

Host: 192.168.56.102

User-Agent: Mozilla/5.0

Accept: text/html,...

Cookie: security=low; PHPSESSID=...

Connection: keep-alive

The server responded with:

HTTP/1.1 200 OK

Server: Apache/2.2.8 (Ubuntu)

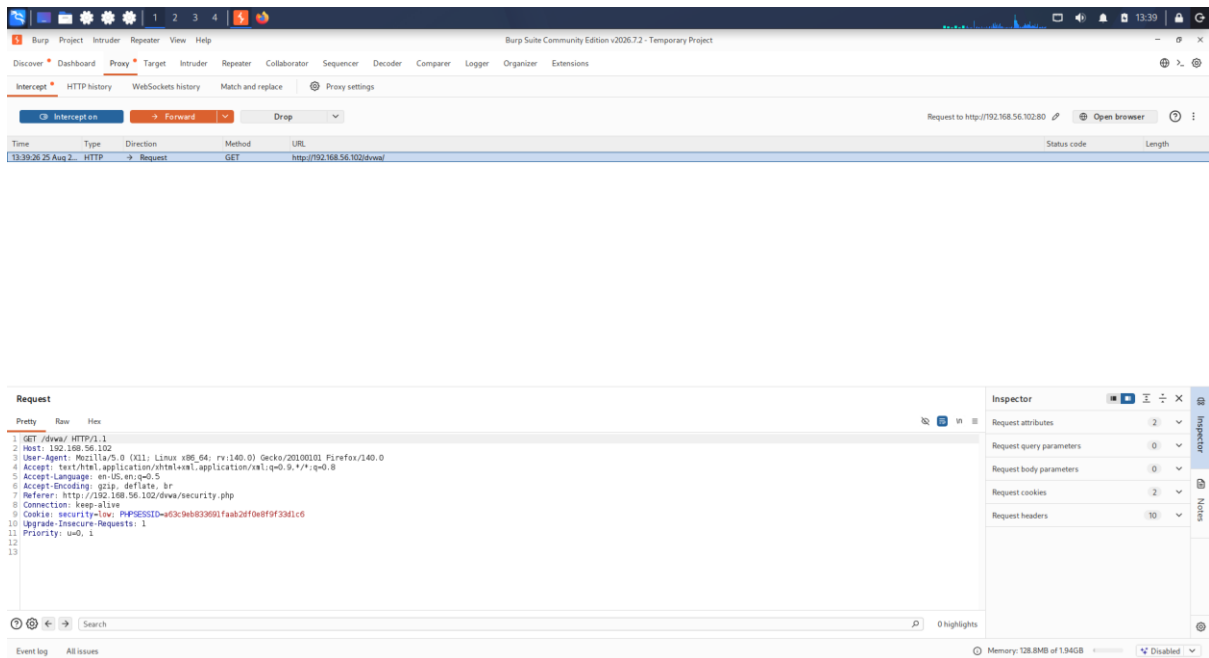
X-Powered-By: PHP/5.2.4-2ubuntu5.10

Content-Type: text/html; charset=utf-8

Observation

The response headers revealed server and PHP version information.

This is useful from a security-assessment perspective because excessive technology/version disclosure can provide information to an attacker during reconnaissance.



6. SQL Injection Testing

6.1 Vulnerability

The DVWA **SQL Injection** module was selected.

The page contains a:

User ID

input field.

The application accepts the supplied value and uses it to retrieve information from the backend database.

6.2 Normal Request

A normal request was first generated using a standard numeric ID.

The resulting application response displayed information associated with the supplied user ID.

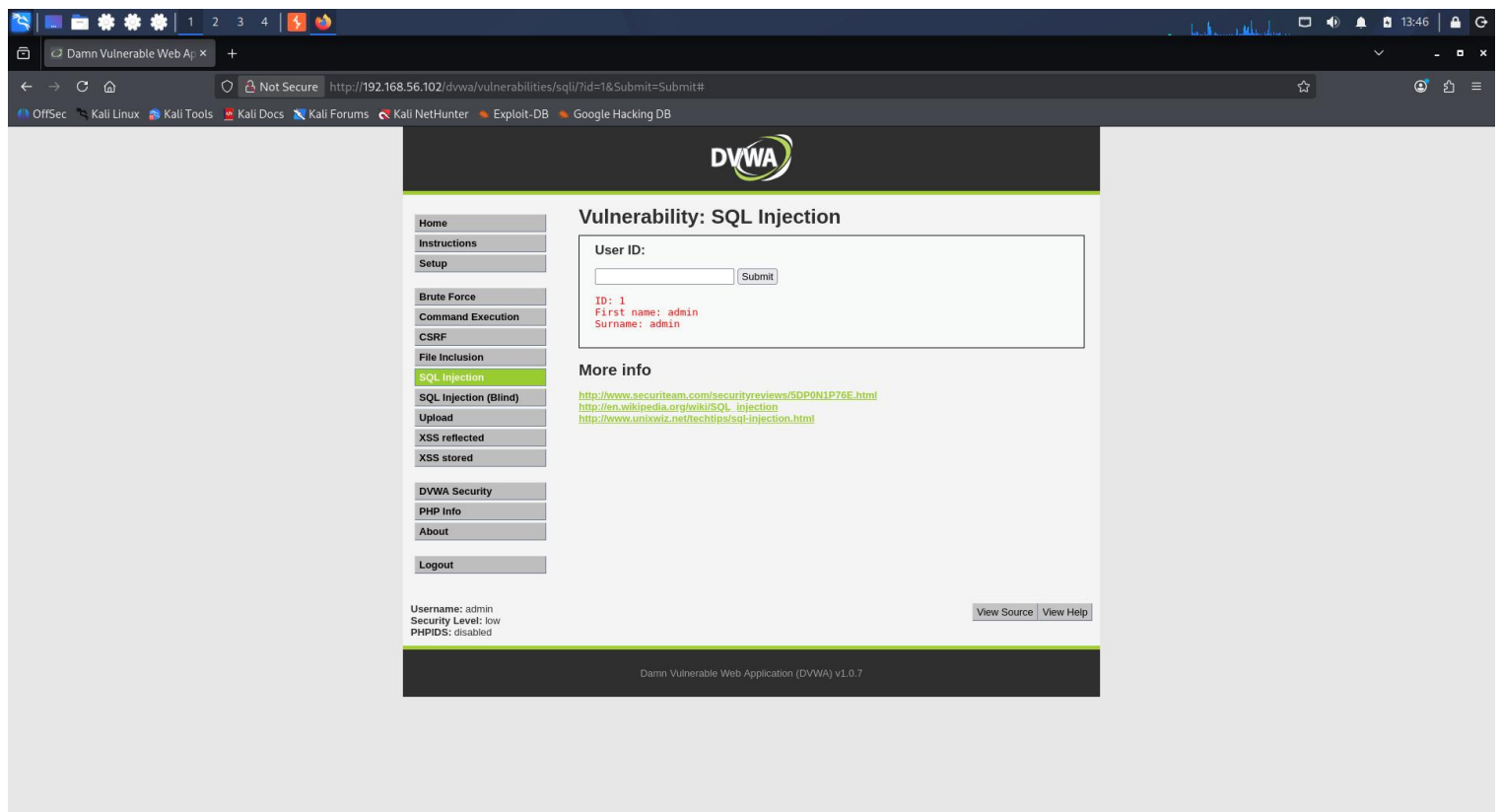
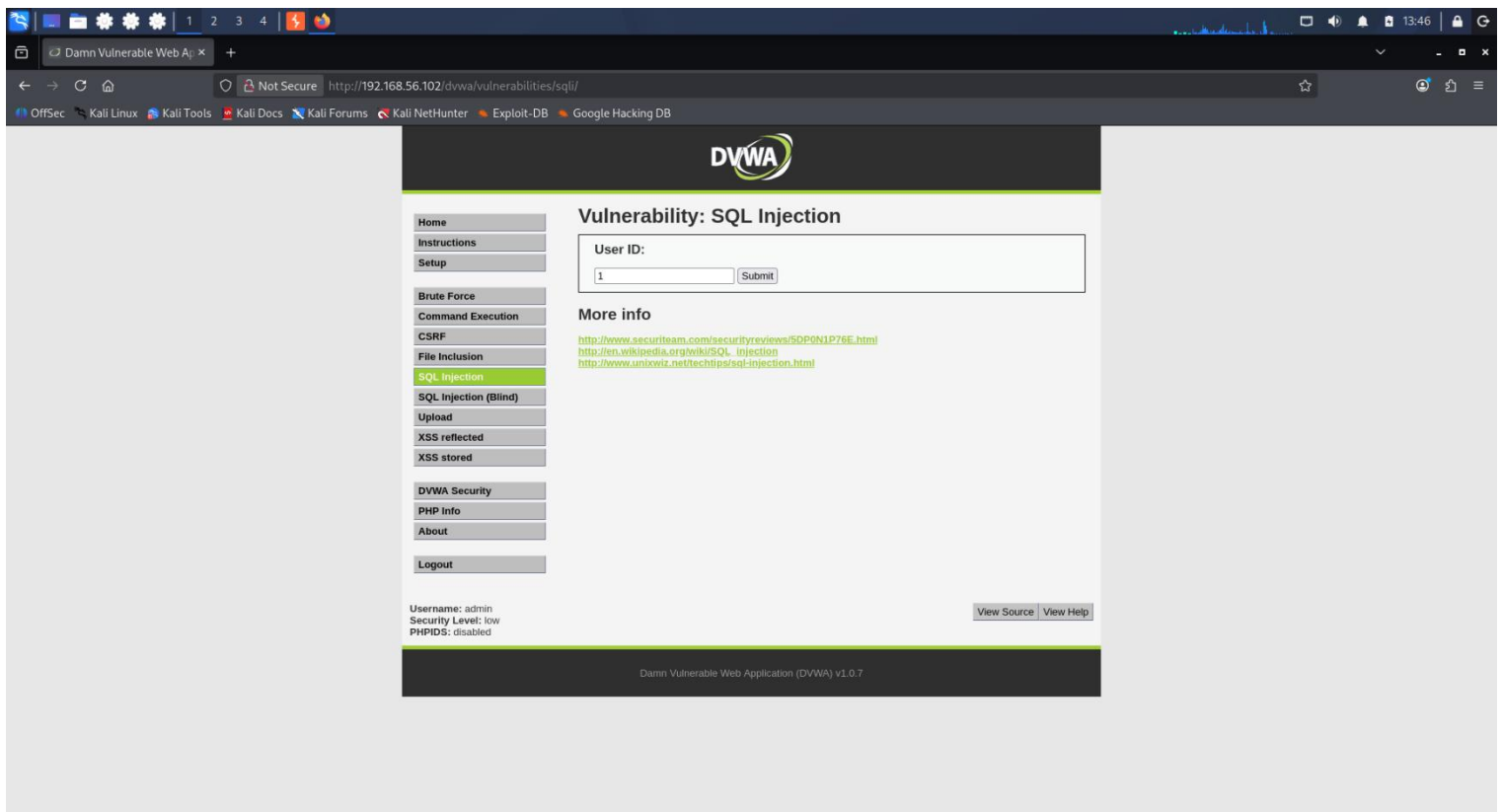
Example:

ID: 1

First name: admin

Surname: admin

This established the baseline behavior of the application.



7. SQL Injection — Low Security

The DVWA security level was set to:

Low

A SQL Injection test was then performed against the User ID parameter.

The test input used was:

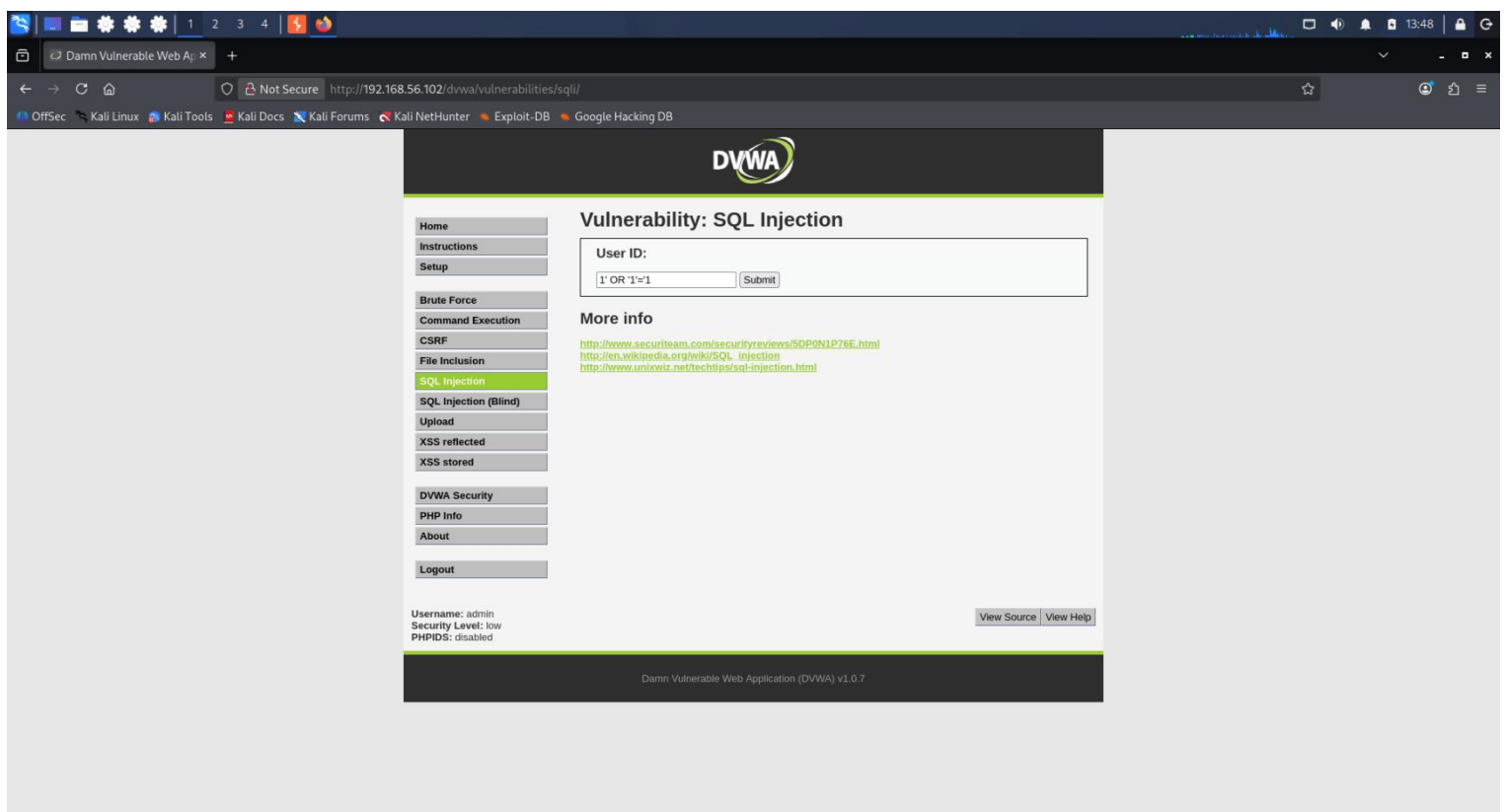
1' OR '1'='1

The resulting request was visible in the browser URL and subsequently captured by Burp Suite.

The request observed in Burp was conceptually:

GET /dvwa/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit
HTTP/1.1

The encoded characters represent the SQL-related input being transmitted as part of the id parameter.



8. SQL Injection Result

The application returned multiple database records rather than only the expected record associated with a single ID.

The observed results included entries such as:

ID: 1' OR '1'='1

First name: admin

Surname: admin

ID: 1' OR '1'='1

First name: Gordon

Surname: Brown

ID: 1' OR '1'='1

First name: Hack

Surname: Me

ID: 1' OR '1'='1

First name: Pablo

Surname: Picasso

ID: 1' OR '1'='1

First name: Bob

Surname: Smith

Result

This demonstrated that the application was vulnerable to SQL Injection at the **Low** security level.

Instead of treating the input strictly as a user ID, the application allowed the supplied input to influence the SQL query.

Finding

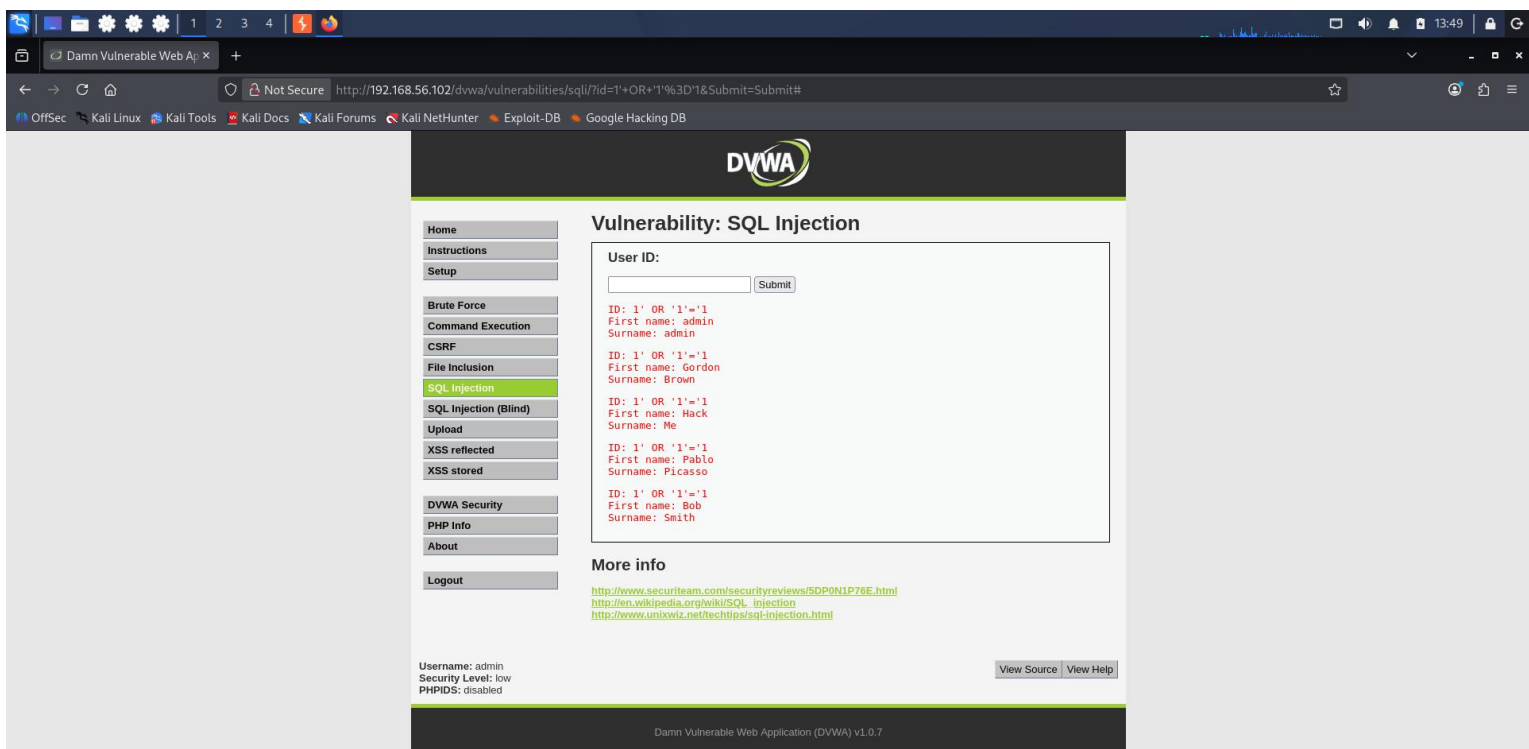
SQL Injection — Confirmed

Severity: High

Affected component: DVWA SQL Injection module

Affected parameter:

Id



9. Burp Suite Evidence

Burp Suite HTTP History recorded the SQL Injection request.

The captured request showed:

GET /dvwa/vulnerabilities/sqli/

with the manipulated id parameter.

The corresponding server response returned HTML containing multiple database records.

This provided both:

Request-side evidence

The attacker-controlled parameter was visible in the HTTP request.

Response-side evidence

The server returned multiple database records in response to the manipulated input.

This makes the Burp evidence particularly useful for the final report because it demonstrates the vulnerability at the HTTP level rather than relying only on the browser screenshot.

The screenshot displays the Burp Suite Community Edition v2026.7.2 interface. The top menu bar includes options like Discover, Dashboard, Proxy, Target, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, and Extensions. The main window is divided into three panes. The left pane shows the HTTP history table with columns for #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, Notes, TLS, IP, Cookies, Time, Listener port, and Start response. The middle pane shows the details of the selected request (ID 10), including its raw text and hex representation. The right pane shows the details of the selected response (ID 1), including its raw text and hex representation. The response is an HTML document from 'Damn Vulnerable Web App (DVWA) v1.0.7'.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/			302	489	HTML					192.168.56.102	PHPSESSID=32079bf...	13:54:13 25 Au...	8080	46
2	http://192.168.56.102	GET	/dvwa/login.php			200	1664	HTML	php	Damn Vulnerable Web App ...			192.168.56.102	security=high	13:54:15 25 Au...	8080	30
3	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/			302	397	HTML					192.168.56.102		13:54:21 25 Au...	8080	26
4	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/			302	397	HTML					192.168.56.102		13:54:22 25 Au...	8080	36
5	http://192.168.56.102	GET	/dvwa/login.php			200	1663	HTML	php	Damn Vulnerable Web App ...			192.168.56.102	security=high	13:54:38 25 Au...	8080	30
6	http://192.168.56.102	POST	/dvwa/login.php			302	392	HTML	php	Damn Vulnerable Web App ...			192.168.56.102		13:54:51 25 Au...	8080	32
7	http://192.168.56.102	GET	/dvwa/index.php			200	4932	HTML	php	Damn Vulnerable Web App ...			192.168.56.102		13:54:53 25 Au...	8080	23
8	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/			200	4684	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:54:59 25 Au...	8080	43
9	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/?id=1&Submit=5...			200	4739	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:55:21 25 Au...	8080	33
10	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/?id=1%27+OR+%...			200	4684	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:55:30 25 Au...	8080	36
11	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/			200	4684	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:55:53 25 Au...	8080	35
12	http://192.168.56.102	GET	/dvwa/vulnerabilities/sqli/?id=1%27+OR+%...			200	4684	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:56:18 25 Au...	8080	28

The selected request (ID 10) is a GET request to `/dvwa/vulnerabilities/sqli/?id=1%27+OR+%...`. The selected response (ID 1) is an HTTP 200 OK response from the server, containing HTML content.

10. SQL Injection — Medium Security

The DVWA security level was subsequently changed from:

Low to:

Medium

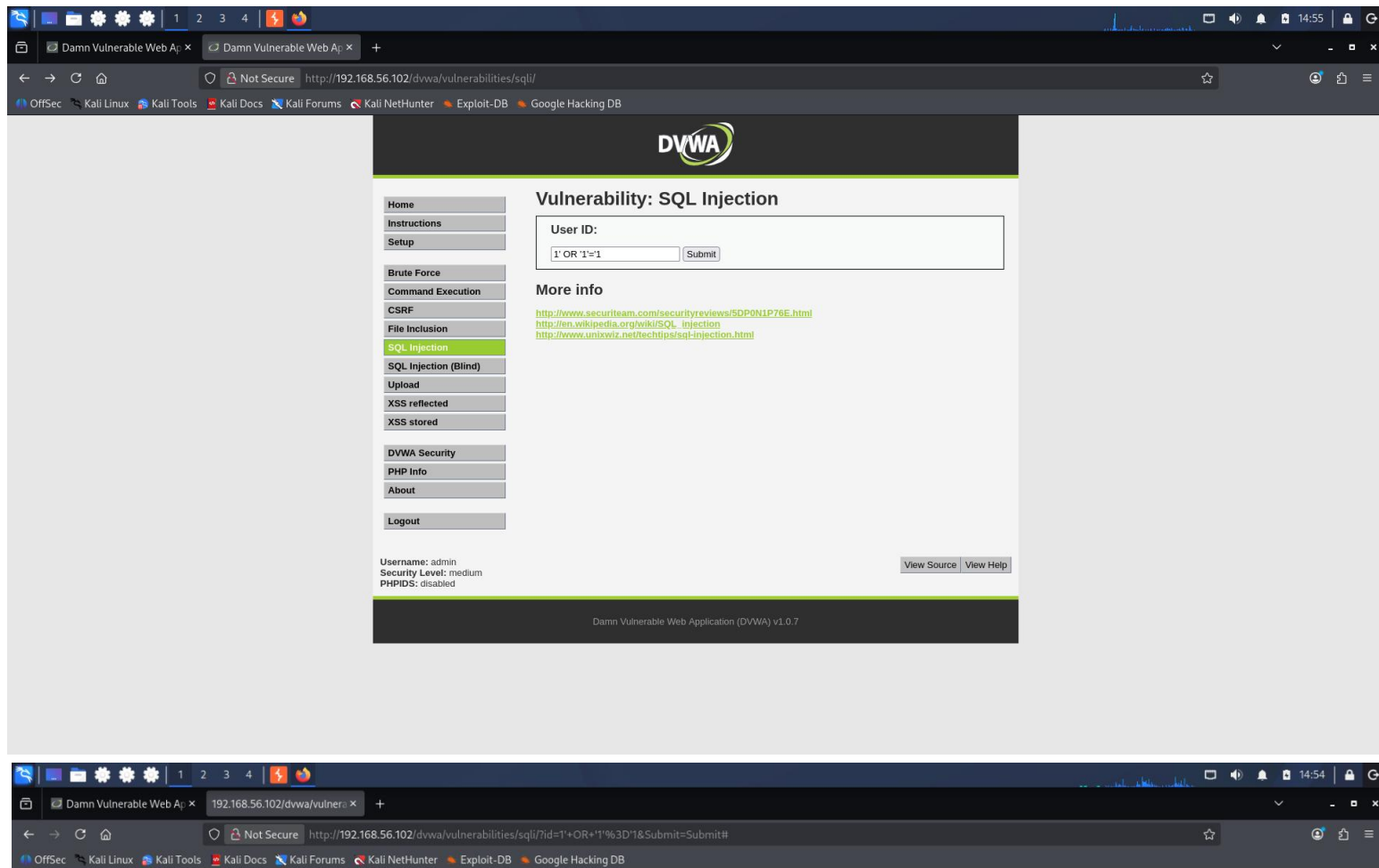
The application confirmed:

Security Level: medium

The same SQL Injection functionality was then tested again.

The application behaved differently from the Low-security configuration.

This demonstrated that DVWA's security level changes the way the vulnerable input is processed.



11. SQL Injection Error Analysis

During testing at a higher security configuration, an SQL-related error was observed.

The browser displayed an error similar to:

You have an error in your SQL syntax;

check the manual that corresponds to your MySQL server version

for the right syntax to use...

Significance

This is important evidence because the application exposed an internal database error directly to the user.

The error reveals:

- The backend uses MySQL.
- The application is constructing SQL queries using the supplied input.
- Database syntax errors are being exposed to the client.

This can be classified as **Information Disclosure / Improper Error Handling** in addition to being useful evidence during SQL Injection testing.

12. SQL Injection Security-Level Comparison

The testing performed so far can be summarized as:

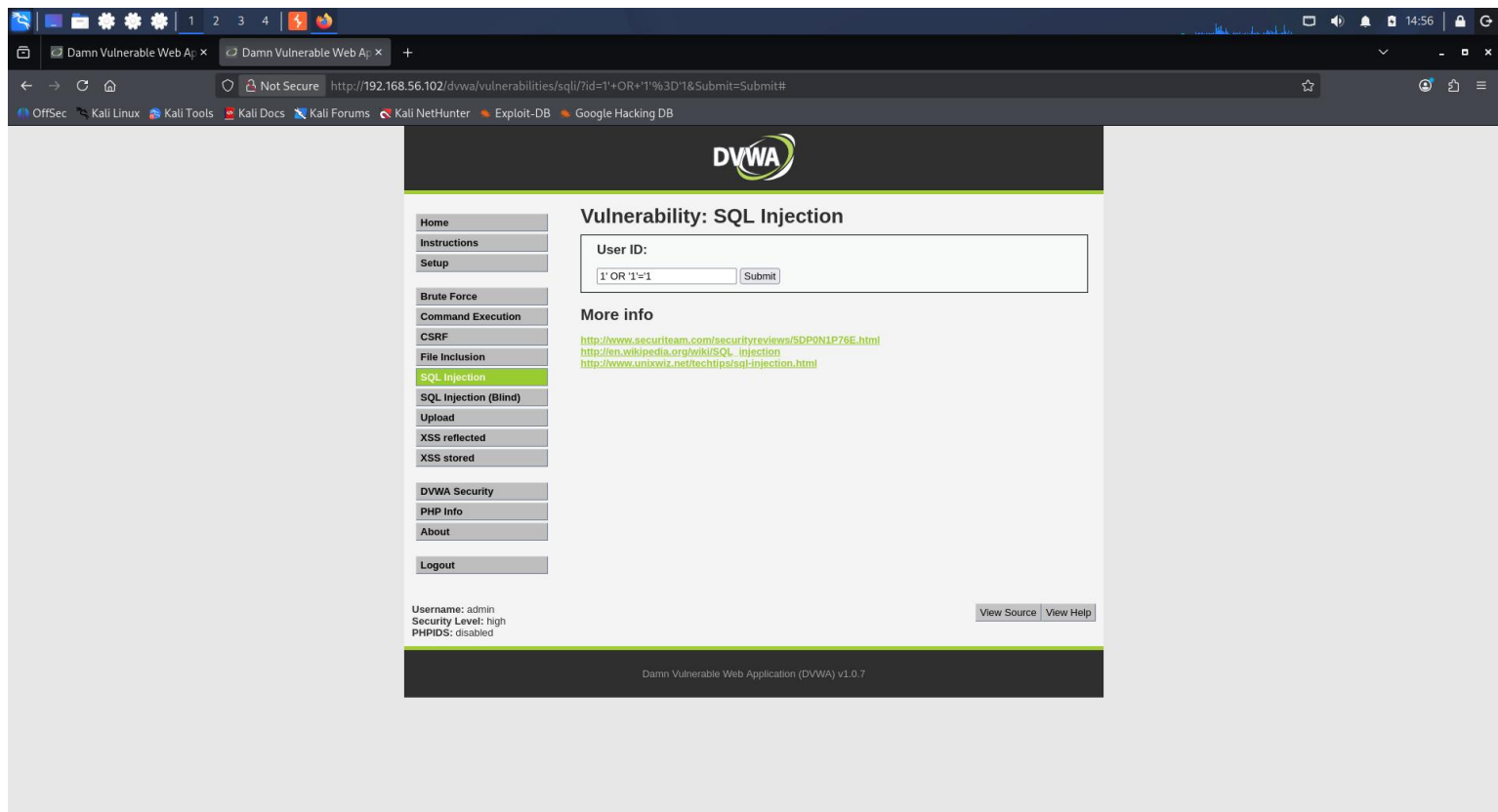
Security Level	Observation
Low	SQL Injection successfully demonstrated
Medium	Application behavior changed and the previous test did not produce the same result
High	Tested as part of the configuration comparison; further detailed testing can be continued later

Important observation

The security level does **not** mean that the underlying application suddenly becomes completely secure.

Rather, DVWA intentionally changes its input handling to demonstrate different levels of protection.

For the report, this is useful because it shows that security controls can alter the application's response to malicious input.



13. Reflected XSS Testing

Another vulnerability tested during Task 2 was:

Reflected Cross-Site Scripting (XSS)

The DVWA Reflected XSS module was accessed.

The vulnerable parameter was: name

The request was intercepted using Burp Suite.

The application accepted user-controlled input through the parameter.

14. Reflected XSS Payload

A simple JavaScript test payload was supplied to the vulnerable parameter.

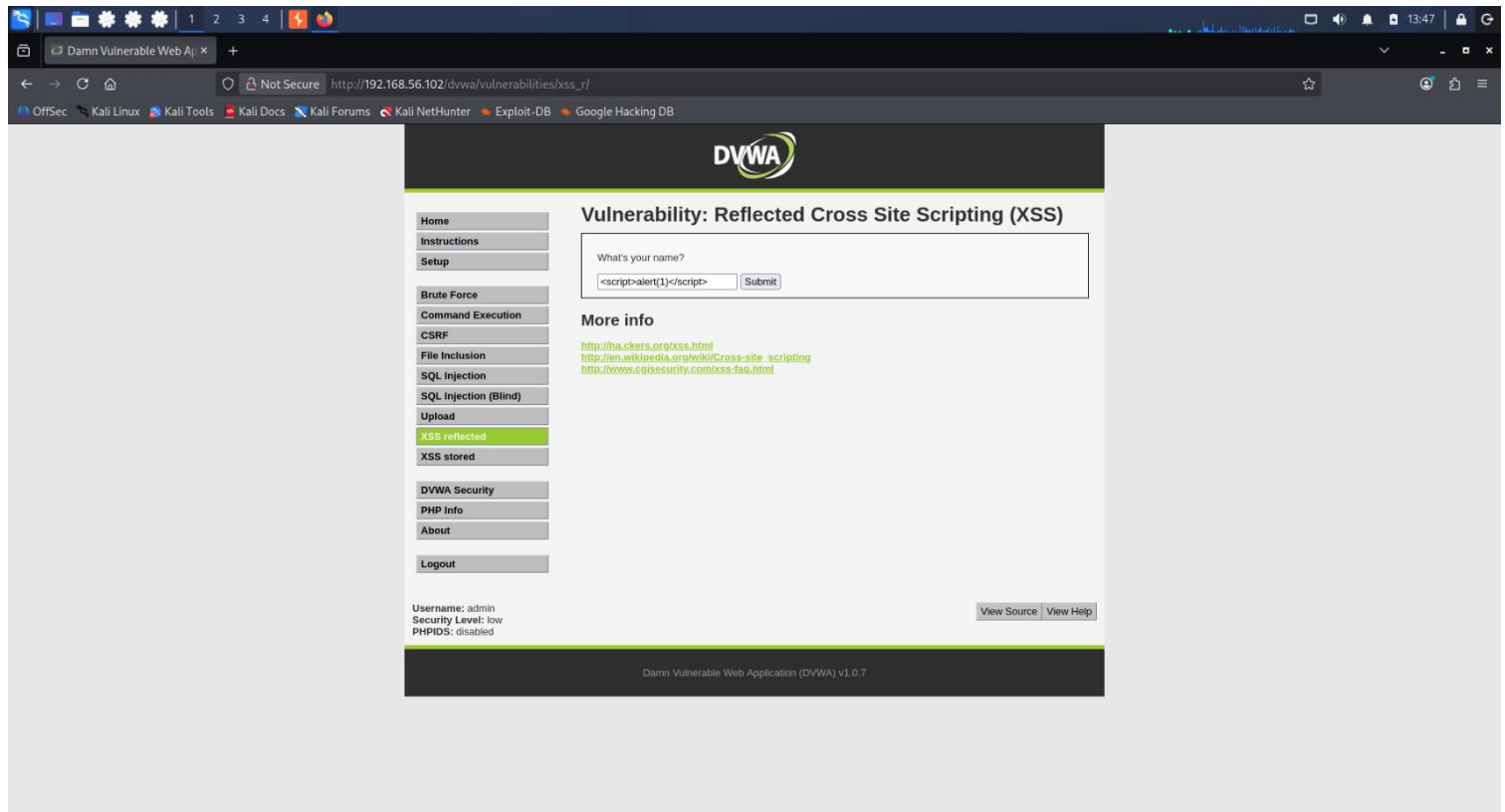
The resulting URL contained the encoded payload.

The browser subsequently displayed a JavaScript alert generated from the supplied input.

Result: The alert demonstrated that the supplied input was reflected into the application's response and interpreted by the browser as executable script.

Therefore:

Reflected XSS was successfully demonstrated.



15. Burp Evidence for Reflected XSS

Burp Suite HTTP History showed the request to:

/dvwa/vulnerabilities/xss_r/

The request contained the user-controlled name parameter.

The corresponding response contained the reflected input.

Burp's response view also showed the HTML structure of the vulnerable page, including the form: `<form ... method="GET">`

and the input: `<input type="text" name="name">`

This provides useful technical evidence that the application accepts user input through the name parameter.

1234

Burp Suite Community Edition v2026.7.2 - Temporary Project

DiscoverDashboardProxyTargetIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensions

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

Pro version only

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response ...
1	http://192.168.56.102	GET	/dvwa/			200	4845	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:39:26 25 Au...	8080	57
2	http://192.168.56.102	GET	/dvwa/vulnerabilities/xss_r/			200	4664	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:40:57 25 Au...	8080	49
3	http://192.168.56.102	GET	/dvwa/vulnerabilities/xss_r/			200	4665	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:41:09 25 Au...	8080	64
4	http://192.168.56.102	GET	/dvwa/vulnerabilities/xss_r/?name=%3Csc...		✓	200	4707	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:41:36 25 Au...	8080	62

Request

Response

Inspector

1234

Burp Suite Community Edition v2026.7.2 - Temporary Project

DiscoverDashboardProxyTargetIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensions

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

Pro version only

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response ...
1	http://192.168.56.102	GET	/dvwa/			200	4845	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:39:26 25 Au...	8080	57
2	http://192.168.56.102	GET	/dvwa/vulnerabilities/xss_r/			200	4664	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:40:57 25 Au...	8080	49
3	http://192.168.56.102	GET	/dvwa/vulnerabilities/xss_r/			200	4665	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:41:09 25 Au...	8080	64
4	http://192.168.56.102	GET	/dvwa/vulnerabilities/xss_r/?name=%3Csc...		✓	200	4707	HTML		Damn Vulnerable Web App ...			192.168.56.102		13:41:36 25 Au...	8080	62

Request

Response

Inspector

16. Reflected XSS Result

Vulnerability

Reflected Cross-Site Scripting (XSS)

Parameter

name

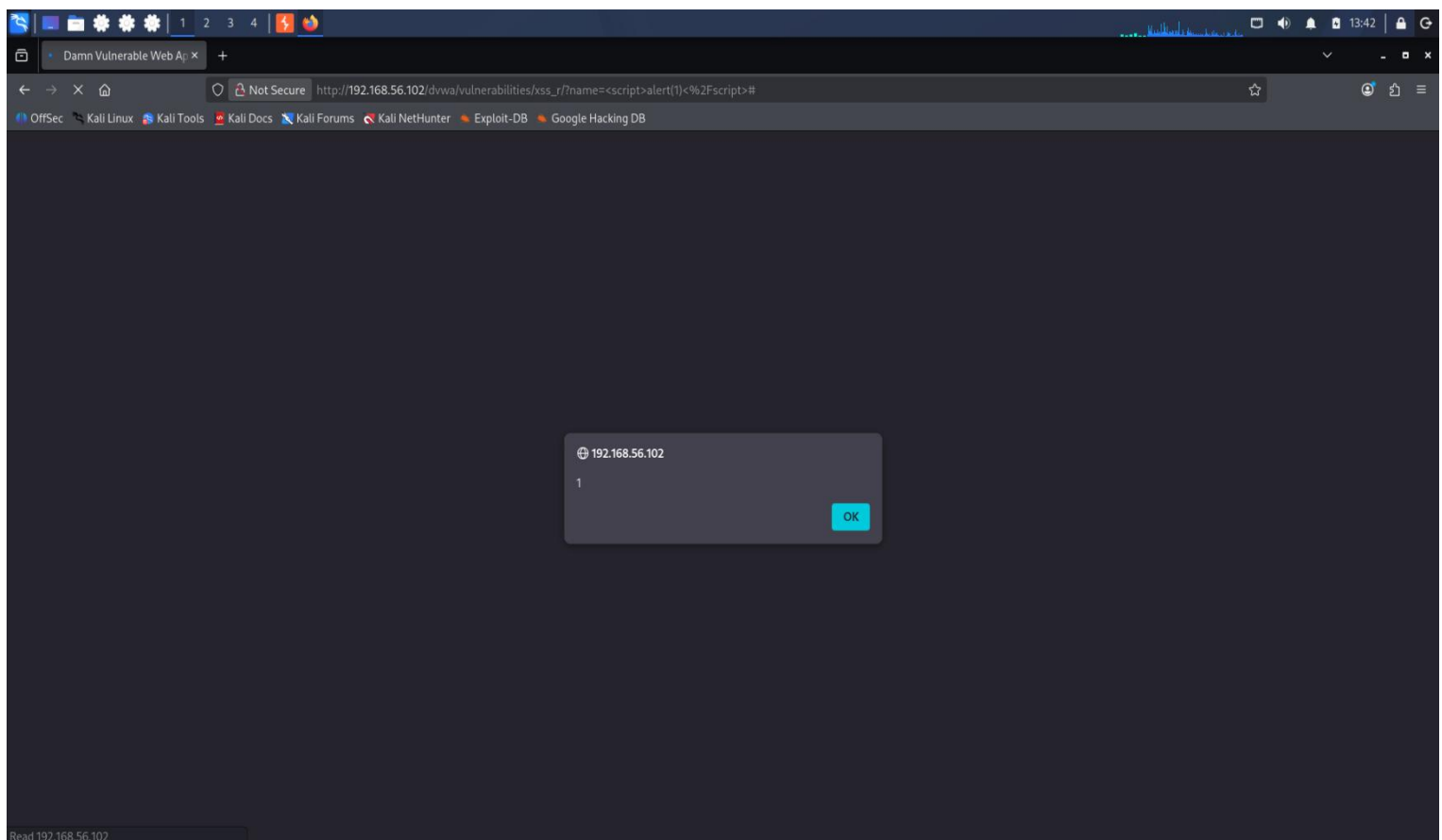
Result

User-supplied script was reflected into the response and executed by the browser.

Severity

Medium

The exact severity can be adjusted in the final report depending on the demonstrated impact and assessment methodology.



17. Information Disclosure

During HTTP analysis, the following response information was observed:

Server: Apache/2.2.8 (Ubuntu)

X-Powered-By: PHP/5.2.4-2ubuntu5.10

The application also exposed a MySQL syntax error during SQL Injection testing.

These observations indicate that the application exposes implementation details that would normally be better hidden from users.

Potential information disclosed

Web server technology

Operating system/platform information

PHP version

Database technology

Database error information

Finding

Information Disclosure / Improper Error Handling

Suggested severity: Low–Medium

18. Findings Identified So Far

ID	Finding	Status	Severity
F-01	SQL Injection	Confirmed	High
F-02	Reflected XSS	Confirmed	Medium
F-03	Information Disclosure through HTTP headers	Observed	Low
F-04	Database Error Disclosure	Confirmed	Medium
F-05	Security-level comparison	Completed	Informational

20. Preliminary Risk Summary

Based on what we've actually completed so far:

SQL Injection

↓

User-controlled ID

↓

SQL query manipulation

↓

Multiple database records returned

↓

Potential unauthorized database access

and:

Reflected XSS

↓

User-controlled name parameter

↓

Input reflected in HTTP response

↓

Browser interprets supplied script

↓

JavaScript executes

21. Remediation

SQL Injection

The application should:

- Use **prepared statements / parameterized queries**.
- Avoid directly concatenating user input into SQL queries.
- Validate input according to the expected data type.
- Apply server-side input validation.
- Use least-privilege database accounts.
- Avoid exposing SQL errors to users.

Recommended approach

Instead of constructing SQL queries using raw user input, applications should use parameterized database queries so that user input is treated as data rather than SQL syntax.

Reflected XSS

The application should:

- Properly encode user-controlled output.
 - Apply context-appropriate HTML/JavaScript escaping.
 - Validate input where appropriate.
 - Implement a suitable **Content Security Policy (CSP)**.
 - Avoid inserting untrusted input directly into HTML.
-

Information Disclosure

The application should:

- Remove unnecessary server/version information.
 - Disable detailed errors in production.
 - Use generic error messages for users.
 - Record detailed errors in protected server-side logs.
-